Bonus Tips for Success:

Understand the basics thoroughly: Fundamental concepts like component lifecycle, hooks, state management, and JSX are frequently tested.

Focus on problem-solving: Be prepared to write code or explain your thought process for building a component, fixing a bug, or implementing a feature.

Practice debugging: You may be given broken code and asked to fix it. Familiarize yourself with React's developer tools.

Be prepared for follow-up questions: Most interviewers will ask deeper questions based on your answers, so aim for clarity and understanding in your explanations.

Communication is key: During coding tasks, communicate your thought process clearly. This helps interviewers understand how you approach problems.

50+ Practical and Scenario-based React Interview Questions

1. Build a simple to-do list app with React.

Requirements:

Allow users to add new tasks.

Mark tasks as complete or delete them.

Save tasks to local storage so they persist on page reload.

2. Explain how to fetch data from an API using hooks in React.

Bonus: How would you handle loading states and errors in your component?

3. How would you optimize a React application for performance?

Discuss optimizations like code splitting, memoization, lazy loading, etc.

4. Explain the difference between controlled and uncontrolled components.

Demonstrate with examples of forms in React.

5. How would you implement form validation in a React app?

Use either built-in hooks or third-party libraries like Formik or React Hook Form.

6. What is prop drilling, and how can you avoid it?

Provide an example where prop drilling occurs, then show how to use Context API or a state management library (like Redux) to avoid it.

7. How do you manage the global state in a React application?

Compare Context API with Redux and describe when to use each approach.

8. Implement a search filter in a React app.

Given a list of items, create a filter input that dynamically displays items matching the search term.

9. Explain how React's use effect hook works, and describe its common use cases.

Follow up: How would you cancel a fetch request inside useEffect to prevent memory leaks?

10. How would you implement pagination in a React app?

Given an API that returns a large dataset, demonstrate how to fetch and display data in pages.

11. Describe how to handle authentication in a React application.

Discuss approaches like JWT tokens, OAuth, or session-based authentication.

12. How would you implement a dynamic theme switcher (e.g., light/dark mode) in a React app?

Include saving user preference in local storage so that it persists across sessions.

13. How would you prevent unnecessary re-renders in a React component?

Demonstrate the use of React.memo, useMemo, and useCallback hooks.

14. How does server-side rendering (SSR) work with React, and what are its benefits?

Mention frameworks like Next.js, which simplify SSR for React applications.

15. Create a reusable table component in React.

Requirements:

- Accepts data and column configuration as props.
- Features sorting and filtering.

16. Explain the purpose of React keys and why they are important when rendering lists.

Discuss the importance of key props in ensuring correct rendering and efficient updates.

17. How would you handle error boundaries in a React application?

Implement a component that catches JavaScript errors anywhere in the component tree and displays a fallback UI.

18. Demonstrate how to lazy load components in React.

Explain React.lazy and Suspense for code-splitting and loading large components dynamically.

19. How would you implement route protection in React (e.g., redirecting unauthorized users)?

Use React Router for route management and demonstrate how to create protected routes.

20. Explain how context and reducers work together in React.

Implement a small example where global state is managed using the Context API and the useReducer hook.

21. How would you manage a modal component in React?

Implement a modal component with open/close functionality, which can be reused across the app.

22. How would you test React components?

Explain the use of tools like Jest and React Testing Library for unit and integration testing of React components.

23. How do you deal with stale closures in React, especially in the context of useEffect?

Explain how closures in React hooks can lead to stale values and how to resolve them by using updated dependencies or useRef.

24. How would you implement infinite scrolling in a React application?

Explain how to load more data automatically as the user scrolls to the bottom of the page.

25. What are some common accessibility issues in React applications, and how would you address them?

Demonstrate how to make React apps more accessible by following best practices like using semantic HTML, ARIA roles, and handling focus states.

Advanced Scenario-Based React Interview Questions

1. Scenario: Handling Large Lists Efficiently

Question: You need to render a list of 10,000 items in a React app. Rendering all at once causes performance issues. How would you improve the performance while ensuring smooth user experience?

Answer Expected: Virtualization using libraries like react-window or react-virtualized, which renders only visible items and lazily loads more as the user scrolls.

2. Scenario: Component Performance Degradation

Question: A React component renders too frequently, causing performance issues. You've identified the root cause is re-rendering due to prop changes. How would you prevent unnecessary re-renders?

Answer Expected: Use React.memo() for functional components to prevent unnecessary updates, PureComponent for class components, useCallback for memoizing functions, and useMemo for expensive computations. Explain dependency management in hooks.

3. Scenario: State Management in a Large Application

Question: Your React app is growing large, and managing state at the component level is becoming cumbersome. What state management approach would you use for scalability, and why?

Answer Expected: Discuss using Redux, Context API with useReducer, or even more modern solutions like Zustand or Recoil. Explain how to structure state across large applications, the pros and cons of each method, and when to use them.

4. Scenario: Server-Side Rendering (SSR) and SEO

Question: You have a React app that needs better SEO and faster initial load times. How would you convert it to use server-side rendering (SSR)?

Answer Expected: Use a framework like Next.js to implement SSR. Explain how SSR improves SEO by serving pre-rendered HTML, improves time-to-first-byte (TTFB), and discuss the trade-offs between SSR and client-side rendering.

5. Scenario: Error Handling in Large React Apps

Question: A critical feature in your app occasionally crashes without throwing explicit errors. How would you handle such errors and ensure the rest of the app remains functional?

Answer Expected: Implement error boundaries in React using componentDidCatch or Error Boundary components. Discuss how to capture errors in child components and render fallback UI while maintaining app stability.

6. Scenario: Handling Asynchronous Operations

Question: Your React component needs to fetch data from multiple APIs in sequence. However, one of the APIs is slow and sometimes fails. How would you handle this gracefully in your component?

Answer Expected: Discuss handling promises with async/await and using error handling methods such as try/catch. Consider adding retry logic with exponential backoff for the failing API, and explain how to handle loading and error states with useState or custom hooks.

7. Scenario: Migrating to Hooks from Class Components

Question: You have a large codebase using class components and lifecycle methods. The team decides to refactor everything to functional components using hooks. How would you approach this migration without breaking existing functionality?

Answer Expected: Discuss how to safely refactor class components to functional components incrementally. Explain mapping lifecycle methods (like componentDidMount, componentDidUpdate) to hooks like useEffect, and ensure tests are maintained or improved during the migration.

8. Scenario: Handling Complex Forms

Question: Your app has a complex multi-step form with dynamic fields and validation rules that change based on user input. How would you manage form state, validation, and submission?

Answer Expected: Use form management libraries like Formik or React Hook Form for efficient form state management. Discuss how to conditionally render fields, dynamically change validation schemas (e.g., with Yup), and manage form steps using local state or context.

9. Scenario: Real-time Data Updates

Question: You need to build a chat application where the UI should automatically update with new messages as soon as they arrive from the server. How would you implement real-time data updates in React?

Answer Expected: Use WebSockets, or services like Socket.io, to establish real-time communication between the client and server. Discuss how to manage live updates in the state using useEffect and how to ensure efficient rendering of new messages without rerendering the entire chat history.

10. Scenario: Implementing Internationalization (i18n)

Question: Your product needs to support multiple languages, and translations are dynamically loaded based on user preferences. How would you implement internationalization in your React app?

Answer Expected: Use libraries like react-i18next or react-intl for managing translations. Explain how to structure translation files, dynamically load translation resources based on

locale, and manage state to switch between languages efficiently.

11. Scenario: Routing in a Complex Application

Question: You're working on a React app with nested routes, protected routes, and dynamic routes (based on user roles). How would you handle such a routing system in React?

Answer Expected: Use React Router to set up nested and dynamic routes. Discuss how to implement route protection using higher-order components (HOCs) or render props to check user authentication and authorization before allowing access to certain routes.

12. Scenario: Client-Side Caching

Question: You need to cache API responses to improve performance and reduce server load in your React app. How would you implement client-side caching?

Answer Expected: Implement caching at the component level using useMemo or useSWR (a React hook for caching and revalidating data). Explain how to store cache data in localStorage or IndexedDB for persistence, and how to invalidate cache when necessary.

13. Scenario: Code-Splitting and Lazy Loading

Question: Your React app is becoming large, and you need to optimize the initial load time. How would you implement code-splitting and lazy loading?

Answer Expected: Use React.lazy and Suspense to dynamically load components only when needed. Discuss webpack's support for code-splitting and explain how to chunk large bundles to improve the initial loading speed.

14. Scenario: Optimizing a Dashboard with Multiple Widgets

Question: You're building a complex dashboard that contains multiple widgets, each fetching data independently. The dashboard is slow to load due to too many network requests happening simultaneously. How would you optimize it?

Answer Expected: Implement techniques like batching API requests or using a global cache for common data (e.g., with useSWR or Redux). Consider lazy loading widgets or using React's Suspense to defer loading until data is ready.

15. Scenario: Managing Complex Animations

Question: You need to implement complex UI animations in your React app (e.g., a draggable list with smooth transitions). How would you achieve this?

Answer Expected: Use libraries like React Spring or Framer Motion for complex and declarative animations. Discuss how to manage animations in React in a performant way and how to handle events like drag-and-drop.

16. Scenario: Improving App Performance in a Slow Network

Question: Your React app performs well in ideal network conditions, but when tested on slow networks, it takes too long to load. How would you handle performance optimizations for users on slow networks?

Answer Expected: Discuss strategies like lazy loading non-essential components, reducing bundle size through code splitting, prioritizing critical resources, using srcset for images, preloading important resources, and employing service workers for offline capabilities.

17. Scenario: Third-Party Library Integration

Question: You're tasked with integrating a third-party library that manages a complex charting system in a React app. However, the library isn't designed with React in mind. How would you handle this integration?

Answer Expected: Use useEffect and refs to initialize and interact with the third-party library's DOM manipulation. Explain how to clean up side effects and manage dependencies without causing memory leaks or performance issues.

18. Scenario: Handling a Memory Leak

Question: Your application shows increased memory usage over time, and after investigation, you suspect a memory leak. How would you debug and fix the memory leak in a React app?

Answer Expected: Use Chrome DevTools or React Profiler to identify memory leaks. Investigate unmounted components that still hold onto state or subscriptions, and ensure you properly clean up in useEffect hooks and component lifecycle methods. Here's the next set of tricky, scenario-based React interview questions that build on the previous ones. These are designed to test your deep understanding of React concepts, architecture, performance optimization, and real-world problem-solving skills.

19 Scenario: Circular Dependencies in State Management

Question: You've noticed circular dependencies between actions and reducers in your Redux setup, leading to confusing state changes and hard-to-track bugs. How would you detect and fix circular dependencies in a React app using Redux?

Tricky Answer Expected: Discuss detecting circular dependencies using tools like madge. Break circular dependencies by restructuring the Redux logic, either by decoupling reducers, splitting up large reducers into smaller ones, or introducing middleware (like Redux-Saga or Thunk) to manage asynchronous logic.

20. Scenario: Handling State with Race Conditions

Question: You have multiple API calls within a single useEffect hook, and race conditions are causing unexpected behavior when one API call returns before the other. How do you ensure

that the state updates occur in the correct sequence?

Tricky Answer Expected: Use Promise.all to handle multiple asynchronous calls and ensure they resolve before updating the state. Alternatively, chain API calls sequentially using async/await. Also, explain how to cancel ongoing requests using AbortController to prevent stale updates.

21. Scenario: Debugging React Component Memory Leaks

Question: After an extensive user session in your app, you notice a significant memory leak due to event listeners not being cleaned up properly. How would you find and fix memory leaks in functional components?

Tricky Answer Expected: Use useEffect's cleanup function to remove event listeners when the component unmounts. Also, mention using the React Profiler in Chrome DevTools to detect memory leaks and ensure event listeners or subscriptions are cleaned up appropriately.

22. Scenario: Dynamic Theming with Context API and Performance Concerns

Question: You are building a dynamic theme switcher using the React Context API. However, you've noticed that changing the theme re-renders large portions of the app, leading to performance degradation. How do you optimize this?

Tricky Answer Expected: Split context into smaller, more specific providers to limit the re-renders. Use React.memo for memoization and consider useContextSelector to selectively update components that rely on specific parts of the context.

23. Scenario: Persisting State with Redux and React Router

Question: You are using React Router for navigation and Redux for state management. When users navigate between pages, they expect form data (stored in Redux) to persist. However, when they refresh the page, the form data disappears. How do you persist Redux state across page reloads?

Tricky Answer Expected: Use redux-persist to store Redux state in localStorage or sessionStorage. Explain how to configure the persistReducer and persistStore, and handle sensitive data by encrypting it before persisting.

24. Scenario: Managing State with Complex Components

Question: You're managing complex state for a large form with multiple fields, some of which are conditionally rendered based on user input. How would you optimize the performance of this form without causing unnecessary re-renders of individual fields?

Tricky Answer Expected: Use React.memo to prevent re-renders of components whose props haven't changed. Consider managing field-specific state in isolated components to avoid triggering re-renders of the entire form when only a few fields are updated. Additionally, use libraries like React Hook Form or Formik for efficient form handling.

25. Scenario: Avoiding Prop Drilling Without Using Context API

Question: You have deeply nested components, and you want to avoid prop drilling but don't want to use Context API due to performance concerns. What alternatives would you suggest?

Tricky Answer Expected: Use component composition or higher-order components (HOCs) to pass functions or data down the tree. Alternatively, use third-party state management solutions like Redux or Zustand, which avoid the overhead of Context API and manage global state efficiently.

26. Scenario: Optimizing SSR in React Applications

Question: You've implemented server-side rendering (SSR) in your React app, but the initial load time is still slow. What would you do to further optimize the performance of your SSR setup?

Tricky Answer Expected: Implement code-splitting on the server using dynamic imports (React.lazy). Preload critical resources using <link rel="preload"> tags. Consider caching rendered HTML or using a Content Delivery Network (CDN) to serve static assets faster. Additionally, optimize the backend by rendering only critical components on the server and deferring non-essential parts to the client.

27. Scenario: Debugging Hydration Issues in SSR

Question: After implementing SSR in your app, you notice "hydration" warnings when React attempts to re-render components on the client-side. How do you resolve this mismatch between the server-rendered HTML and the client-side render?

Tricky Answer Expected: Ensure consistent output on both server and client by managing dynamic content properly (e.g., avoiding Math.random() or Date.now() on the server). Use useEffect for client-side-only behavior. Also, consider using conditional logic to ensure that content doesn't differ between the server and client during hydration.

28. Scenario: Handling Large State with Context API

Question: You're using React's Context API to manage global state, but the state object is growing large, making it hard to maintain and causing performance issues due to excessive re-renders. How do you refactor this setup?

Tricky Answer Expected: Split the context into smaller, more focused contexts (e.g., AuthContext, ThemeContext) to localize state management. Use the useReducer hook inside contexts to manage state changes efficiently. Discuss implementing selectors or leveraging libraries like react-context-selector to minimize re-renders.

29. Scenario: Managing Concurrent Requests in React

Question: You have multiple API requests happening in parallel inside a useEffect hook, and the UI is updated based on the results of these requests. However, if one request fails, the entire UI breaks. How do you manage concurrent requests while handling errors gracefully?

Tricky Answer Expected: Use Promise.allSettled() to handle multiple requests and capture both resolved and rejected promises without breaking the UI. Ensure partial UI updates for successful requests while handling errors gracefully by showing error messages only for the failed requests.

30. Scenario: Optimizing Complex Animations

Question: You're implementing complex animations in your app (e.g., transitions between pages, component mounts/unmounts), but these animations are causing performance bottlenecks on low-end devices. How would you improve the performance of these animations?

Tricky Answer Expected: Use GPU-accelerated CSS animations wherever possible, such as transform and opacity. Offload expensive JavaScript calculations to web workers if necessary. Use libraries like Framer Motion or React Spring to optimize performance and manage animation frames efficiently.

31. Scenario: Handling Browser Compatibility Issues

Question: Your React app behaves differently in certain older browsers (e.g., IE11), causing layout and functionality issues. How do you ensure cross-browser compatibility for your app while maintaining performance and modern best practices?

Tricky Answer Expected: Use polyfills for older browser features (e.g., Babel for ES6+ features). Leverage CSS preprocessors like Autoprefixer to ensure consistent styles across browsers. Employ tools like BrowserStack to test across different browsers and devices. Where needed, conditionally load polyfills or fallbacks for legacy browsers.

32. Scenario: Handling Deep Object Comparisons in React

Question: You have a component that accepts a deeply nested object as a prop. Even when small parts of the object change, the entire component re-renders, causing performance issues. How do you optimize re-rendering for deeply nested objects?

Tricky Answer Expected: Use useMemo or useCallback to memoize the prop if it is a computation. Implement custom comparison logic using React.memo() with a deep comparison function. Alternatively, normalize your state structure (using libraries like normalizr) to handle updates at specific levels of the object.

33. Scenario: Handling Real-Time Updates with Web Workers

Question: You're building a React app that performs heavy computations in the background (e.g., data processing), which slows down the main UI thread. How would you offload this work to

ensure the UI remains responsive?

Tricky Answer Expected: Use Web Workers to offload heavy computations from the main thread. Integrate Web Workers into your React app using libraries like workerize or comlink. Explain how to communicate between the main thread and the Web Worker using postMessage() and onmessage handlers.

34. Scenario: Managing Complex State Updates with Concurrency

Question: You have multiple state updates happening asynchronously inside a single component. How do you ensure that state updates are handled in the correct order without causing inconsistencies, especially in concurrent mode?

Tricky Answer Expected: Use thebatching of state updates provided by React's concurrent mode. Discuss how React's concurrent rendering prioritizes updates and how you can use `useTransition` and `startTransition` to manage complex state updates without blocking the UI.

35. Scenario: Rendering Optimizations for Conditional Components

Question: You have a component that conditionally renders heavy child components based on user input. These components are computationally expensive and slow down the app when rendered. How would you optimize the rendering process?

Tricky Answer Expected: Usecode splitting with `React.lazy()` and `Suspense` to load components only when needed. Alternatively, memoize the components using `React.memo()` and defer their rendering usingskeleton screens orlazy loading for deferred content.

36. Scenario: Handling Global State in Micro-Frontend Architecture

Question: You are working on a micro-frontend architecture, where each micro-app has its own state. However, certain global state (like user data or app-wide settings) needs to be shared across micro-apps. How would you manage and synchronize global state across micro-frontends?

Tricky Answer Expected: Use ashared global state with libraries likeRedux orZustand that can be injected into each micro-app. Discuss usingcustom events orevent buses for cross-app communication, and consider context providers at the root level for sharing global data.

37. Scenario: Handling Asynchronous Rendering and Suspense

Question: Your React app uses `Suspense` for data fetching, but during testing, you find that certain components don't wait for asynchronous data to resolve before rendering, causing layout issues. How would you fix this problem?

Tricky Answer Expected: Ensure you are using`React.Suspense` properly by wrapping components that rely on asynchronous data in a `<Suspense>` boundary. Discuss

implementingReact's concurrent features, using aresource wrapper for data fetching, and potentially using acache layer for managing async rendering.

38. Scenario: Optimizing Use of `useEffect` in Large Components

Question: You have a large component with several `useEffect` hooks that depend on multiple state variables and props. This is causing too many re-renders and performance degradation. How would you optimize the use of `useEffect` in such a scenario?

Tricky Answer Expected: Refactor byconsolidating related effects into one `useEffect` block where possible. Memoize expensive computations using`useMemo`, and ensure you pass only relevant dependencies to avoid unnecessary re-execution. Additionally, usedebouncing orthrottling for handlers inside `useEffect` where appropriate.

39. Scenario: Detecting and Handling Stale Closures

Question: You have a functional component that uses asynchronous functions inside `useEffect`, but you encounter issues where old closures lead to incorrect state being used. How would you handle stale closures in React hooks?

Tricky Answer Expected: Useref variables to keep track of the latest state or create acustom hook that captures the latest state reference. Explain how `useRef` can store mutable state that doesn't cause re-renders and how it can prevent issues with stale closures in async functions.

40. Scenario: Handling Infinite Scroll with Complex Data

Question: You need to implement an infinite scroll feature in a list that fetches data from multiple sources and combines them. The challenge is ensuring that the list dynamically loads without duplication or missing items. How would you handle this?

Tricky Answer Expected: Useintersection observers for detecting when to load more data. Ensure data deduplication by maintaining a uniqueID map for the items and merging data from different sources. Implementdebouncing orthrottling for network requests to prevent unnecessary API calls.

41. Scenario: Preventing Memory Leaks with Event Listeners

Question: You have a component that adds event listeners to the DOM during initialization. Over time, you notice that these event listeners accumulate, causing memory leaks. How would you resolve this?

Tricky Answer Expected: Use the `useEffect` cleanup function to remove event listeners when the component unmounts. Ensure you correctly identify and clean up the listeners for bothmounted andunmounted components, and consider using`useRef` to ensure the correct event listener is being tracked.

42. Scenario: Implementing Optimistic UI with Error Handling

Question: You need to implement an optimistic UI update where changes are reflected in the UI before the API response is received. If the API call fails, you must revert the changes. How would you implement this in React?

Tricky Answer Expected: Maintain alocal copy of the state before making the API call. Update the UI immediately and, if the API call fails,revert to the previous state by using `setState` with the locally stored value. Discuss how to handle side effects and rollback cleanly in complex scenarios.

43. Scenario: Managing State Across Multiple Tabs

Question: Your application allows users to open multiple tabs, but changes made in one tab should reflect immediately in other open tabs (e.g., user login/logout). How would you synchronize state across multiple browser tabs?

Tricky Answer Expected: Use`localStorage` events orBroadcastChannel API to synchronize state across tabs. Listen for changes in state in `localStorage` or useweb workers orservice workers for more complex cross-tab communication.

44. Scenario: Handling High-Frequency Updates Efficiently

Question: You're working on a real-time application (e.g., a stock ticker) that receives frequent updates (every second). How would you optimize rendering in React to prevent performance bottlenecks and ensure smooth UI updates?

Tricky Answer Expected: UserequestAnimationFrame to batch updates and ensure smooth animation frames. Use`useMemo` or`useCallback` to memoize expensive calculations and ensure only necessary components are re-rendered. Leverageweb workers to offload non-UI-intensive tasks to avoid blocking the main thread.

45. Scenario: Managing Permissions in Component Rendering

Question: Your application has components that should only be visible or interactive based on user roles and permissions. However, you want to avoid unnecessary re-renders and prop drilling while implementing this feature. How would you manage permissions efficiently?

Tricky Answer Expected: Use ahigher-order component (HOC) orrender props to encapsulate permission logic. Memoize the permission-checking function to avoid re-runs on every render. If using Context, optimize it with selective context updates or useZustand or similar libraries for efficient state management.

46. Scenario: Gracefully Handling API Rate Limits

Question: Your React app integrates with third-party APIs that enforce rate limits. If the app exceeds the rate limit, subsequent requests are rejected. How would you handle and mitigate

this issue in your React app?

Tricky Answer Expected: Implementrequest queuing andthrottling mechanisms to ensure the app doesn't exceed the rate limit. Use tools likeAxios interceptors or custom request handlers to manage retries with exponential backoff. Cache responses when possible to reduce the number of API requests.

47. Scenario: Managing Dynamic Imports in Code Splitting

Question: You are using dynamic imports for code-splitting in your React app. However, some components fail to load when needed due to chunk loading errors (e.g., network issues or server misconfiguration). How would you handle such chunk loading failures?

Tricky Answer Expected: Useerror boundaries to gracefully handle chunk loading errors. Show a fallback UI to inform the user of loading issues and implement retry logic to reload the chunk after a delay. Additionally, discuss preloading critical chunks and usingservice workers for offline support.

48. Scenario: Handling Component Re-renders due to Prop Changes

Question: A child component is re-rendering unnecessarily whenever its parent component re-renders, even though the props haven't changed. How would you debug and resolve this issue?

Tricky Answer Expected: Use`React.memo()` to prevent the child component from re-rendering when the props haven't changed. Debug usingReact DevTools to identify what's causing the re-renders and optimize with`useCallback()` and`useMemo()` where necessary to prevent prop changes from causing unnecessary renders.

To enhance your learning, I invite you to check out my YouTube channel: UI Dev Guide. The channel is dedicated to providing mock interviews for job seekers, helping in career consulting, and practical examples that mostly asked in the interviews
check below playlist for javascript angular and react:

Angular : 🌐 Must Watch Angular interviews

React: 🌐 Must watch React interviews list before going for interview

Javascript Coding :
🌐 JavaScript coding interview questions | JavaScript interview questions and answers

Javascript: 🌐 JavaScript interview questions